

Object Methods, "this" – Simple Hinglish Summary

Source: <https://javascript.info/object-methods>

Main baat

Objects aam taur par real-world cheezein represent karne ke liye bante hain (users, orders, etc.). Real duniya me user **kuch kar bhi sakta hai**: cart se kuch chunna, login, logout. JavaScript me in **actions** ko object ki properties me rakhe **functions** se represent karte hain.

1. Method kya hai?

Jo function object ki property ho, use us object ka *method* kehte hain.

```
javascript

let user = {
  name: "John",
  age: 30
};

user.sayHi = function() {
  alert("Hello!");
};

user.sayHi(); // Hello!
```

Yahan Function Expression se function banakar `user.sayHi` property me assign kiya. Ab `user.sayHi()` se call kar sakte hain.

Pehle se declare kiya function bhi method bana sakte ho:

```
javascript

function sayHi() {
  alert("Hello!");
}

user.sayHi = sayHi;

user.sayHi(); // Hello!
```

Object-oriented programming (OOP): jab hum entities ko objects se represent karke code likhte hain, use OOP kehte hain. Ye ek bada topic hai (sahi entities chunna, unke beech interaction ka architecture, etc.).

Method shorthand

Object literal me method likhne ka chhota syntax hai:

```
javascript

// dono same kaam karte hain
user = {
  sayHi: function() {
    alert("Hello");
  }
};

// shorthand (behtar dikhta hai)
user = {
  sayHi() { // "sayHi: function(){...}" ke barabar
    alert("Hello");
  }
};
```

`function` keyword hata kar sirf `sayHi()` likh sakte hain. Dono me **chhote subtle differences** hain (inheritance se related, aage aayenge), lekin lagbhag har case me shorthand hi preferred hai.

2. Methods me `this`

Method ko aksar **object ke andar ki jaankari** chahiye hoti hai. Jaise `user.sayHi()` ko `user` ka naam chahiye.

Object ko access karne ke liye method `this` keyword use karta hai.

`this` ki value wo object hai jo "dot ke pehle" hai, yaani jis object se method call hua.

```
javascript
```

```
let user = {  
  name: "John",  
  age: 30,  
  
  sayHi() {  
    // "this" = "current object"  
    alert(this.name);  
  }  
  
};  
  
user.sayHi(); // John
```

`user.sayHi()` chalte waqt `this` = `user`.

`this` ki jagah bahar ka variable kyu nahi?

Technically bahar ke variable se bhi object access kar sakte ho:

```
javascript  
  
let user = {  
  name: "John",  
  age: 30,  
  
  sayHi() {  
    alert(user.name); // "this" ki jagah "user"  
  }  
  
};
```

Lekin ye unreliable hai. Agar `user` ko dusre variable me copy kar do (`admin = user`) aur `user` ko kuch aur bana do, to galat object access hoga:

```
javascript
```

```
let user = {
  name: "John",
  age: 30,

  sayHi() {
    alert( user.name ); // error aayega
  }

};

let admin = user;
user = null; // overwrite

admin.sayHi(); // TypeError: Cannot read property 'name' of null
```

Agar `user.name` ki jagah `this.name` likhte, to code chalta.

3. `this` "bound" nahi hota

JavaScript me `this` zyadatar dusri languages jaisa nahi hai. Ise kisi bhi function me use kar sakte hain, chahe wo kisi object ka method na ho. Ye syntax error nahi hai:

```
javascript

function sayHi() {
  alert( this.name );
}
```

`this` ki value runtime par tay hoti hai, context ke hisaab se.

Yahan ek hi function do alag objects me hai, aur `this` alag hai:

```
javascript
```

```

let user = { name: "John" };
let admin = { name: "Admin" };

function sayHi() {
  alert( this.name );
}

// same function ko do objects me use karo
user.f = sayHi;
admin.f = sayHi;

// in calls me `this` alag hai
// function ke andar "this" = "dot ke pehle wala object"
user.f(); // John (this == user)
admin.f(); // Admin (this == admin)

admin['f'](); // Admin (dot ya square brackets, dono chalte hain)

```

Rule simple hai: `obj.f()` call hone par `f` ke chalte waqt `this = obj`.

Bina object ke call: `this == undefined`

Function ko bina object ke bhi call kar sakte hain:

```

javascript

function sayHi() {
  alert(this);
}

sayHi(); // undefined

```

Strict mode me `this` `undefined` hota hai. `this.name` access karoge to error aayega. **Non-strict mode me** `this` **global object** hota hai (browser me `window`). Ye purana historical behavior hai jise `"use strict"` theek karta hai.

Aam taur par aisa call **programming ki galti** hoti hai. Function me `this` ho to use object context me call hone ki ummeed hoti hai.

"Unbound this" ke nateeje

Agar tum dusri language se aaye ho to "bound `this`" ke aadi ho sakte ho, jahan object me define kiye methods me `this` hamesha usi object ko refer karta hai.

JavaScript me `this` **"free"** hai. Iski value **call ke time** tay hoti hai, aur is par nahi ki method kahan declare hua tha, balki is par ki **"dot ke pehle" kaunsa object hai**.

Fayda: ek function ko alag alag objects ke liye reuse kar sakte hain. **Nuksan:** itni flexibility se galtiyan ke chances badh jate hain.

4. Arrow functions me `this` nahi hota

Arrow functions special hain: unka apna `this` nahi hota. Arrow function me `this` use karo to wo **bahar ke "normal" function se** liya jata hai.

```
javascript

let user = {
  firstName: "Ilya",
  sayHi() {
    let arrow = () => alert(this.firstName);
    arrow();
  }
};

user.sayHi(); // Ilya
```

Yahan `arrow()` bahar wale `user.sayHi()` method ka `this` use karta hai. Ye tab kaam aata hai jab hume alag `this` nahi, balki **bahar ke context se** `this` chahiye. (Aage "Arrow functions revisited" chapter me detail hai.)

Summary

- Object property me stored functions ko "**methods**" kehte hain.
- Methods se objects "**act**" kar sakte hain, jaise `object.doSomething()`.
- Methods object ko `this` se refer kar sakte hain.

`this` ki value runtime par tay hoti hai:

- Function declare karte waqt wo `this` use kar sakta hai, lekin us `this` ki **koi value nahi hoti jab tak function call na ho**.
- Function ko objects ke beech **copy** kiya ja sakta hai.
- Function ko `object.method()` syntax se call karo to call ke dauran `this = object`.

Arrow functions special hain: unka `this` nahi hota. Arrow function ke andar `this` bahar se liya jata hai.

Practice Tasks (Answers)

1. Object literal me `this`:

```
javascript
```

```
function makeUser() {
  return {
    name: "John",
    ref: this
  };
}

let user = makeUser();

alert( user.ref.name ); // ?
```

Jawab: Error. `this` set karne ke rules object definition ko nahi dekhte, sirf **call ke moment** ko dekhte hain. `makeUser()` normal function ki tarah call hua (method ki tarah nahi), isliye `this = undefined`. **Poore function ke liye `this` ek hi hota hai**, code blocks aur object literals use nahi badalte.

Ulta case (ye chalega):

```
javascript

function makeUser() {
  return {
    name: "John",
    ref() {
      return this;
    }
  };
}

let user = makeUser();

alert( user.ref().name ); // John
```

Ab `user.ref()` method hai, aur `this` dot ke pehle wala object hai.

2. Calculator object:

```
javascript
```

```

let calculator = {
  sum() {
    return this.a + this.b;
  },

  mul() {
    return this.a * this.b;
  },

  read() {
    this.a = +prompt('a?', 0);
    this.b = +prompt('b?', 0);
  }
};

calculator.read();
alert( calculator.sum() );
alert( calculator.mul() );

```

3. Chaining ((ladder)): Solution: har method se **object khud ((this))** return karo.

javascript

```

let ladder = {
  step: 0,
  up() {
    this.step++;
    return this;
  },
  down() {
    this.step--;
    return this;
  },
  showStep() {
    alert( this.step );
    return this;
  }
};

ladder.up().up().down().showStep().down().showStep(); // 1 phir 0

```

Lambi chain ke liye ek line me ek call bhi likh sakte ho:

```
javascript
```

```
ladder
```

```
.up()  
.up()  
.down()  
.showStep() // 1  
.down()  
.showStep(); // 0
```

Quick Summary

Baat

Yaad rakho

Method

Object ki property me rakha function

Shorthand

```
sayHi() { ... }
```

```
this
```

Dot ke pehle wala object (call ke time tay hota hai)

```
user.name
```

vs

```
this.name
```

```
this.name
```

reliable hai

Bina object ke call

Strict mode me

```
this = undefined
```

Function copy

Ek function kai objects me use ho sakta hai

Arrow function

Apna

```
this
```

 nahi, bahar se leta hai

Chaining

Har method

```
return this;
```

 kare